🐳 Docker + CI/CD Notes (Beginner Friendly)

My personal notes. Written in simple words so future me understands fast.

Contents

1. Docker: what it is
2. When to use / when NOT to use Docker
3. Docker: steps to get started
4. Docker boilerplate (copy and edit)
5. Useful Docker commands
6. CI/CD: what it is
7. When to use / when NOT to use CI/CD
8. CI/CD: steps to set up
9. CI/CD boilerplate (copy and edit)
10. Common errors and fixes
11. My project notes
12. To learn next

---

1. Docker: what it is

The problem: "It works on my laptop but crashes on yours." 😩 Different Node version, missing library, different OS...

Docker's fix: pack your app **and everything it needs** into one sealed box. That box runs the same on every computer.

Think of it like a **lunchbox**: the food (your app) and the spoon, salt, and napkin (libraries, settings) are all inside. You carry it anywhere and it works.

Words you will hear

Word	Easy meaning
Image	The recipe/blueprint. Read-only. You build it once.
Container	A running copy made from an image. The "cooked meal".
Dockerfile	A text file with the steps to build your image.
Registry	Online store for images (Docker Hub, GitHub Container Registry).
Volume	A safe folder that keeps data even after the container is deleted.
Docker Compose	Start many containers together (app + database) with one file.

Image vs Container: image = class, container = object made from it. One image can make many containers.

Container vs Virtual Machine: a VM carries a whole OS (heavy, slow). A container shares your computer's OS (light, starts in seconds).

---

2. When to use / when NOT to use Docker

✅ Use Docker when

- Your team says "works on my machine" a lot
- App needs a database, cache, etc. and you want one-command setup (`docker compose up`)
- You want to deploy to a server or cloud
- You want a clean, same environment for dev, test, and production
- You are running CI/CD pipelines
- You want to try a tool (MongoDB, Redis, Postgres) without installing it on your laptop

❌ Don't use Docker when

- It's a tiny script or a simple static page. Overkill.
- You need direct hardware access and are just starting out (USB, GPIO, Arduino serial). Possible, but needs extra flags and is more painful for beginners.
- You need heavy GUI apps or GPU stuff and don't know the setup yet. Harder to configure.
- Your laptop is low on RAM/disk. Docker eats both.
- You are only learning basic coding. Learn the code first, Docker later.

---

3. Docker: steps to get started

Step 1: Install Docker Desktop (Windows/Mac) or Docker Engine (Linux). Check it works:

```bash
docker --version        # shows the version, means install is OK
docker run hello-world  # downloads a tiny test image and runs it
```

Step 2: Make a `Dockerfile` in your project root (see boilerplate below). This tells Docker how to build your app.

Step 3: Make a `.dockerignore` so junk doesn't get copied into the image.

Step 4: Build the image

```bash
docker build -t myapp .
```

`-t myapp` = give the image a name. The `.` = "use the Dockerfile in this folder".

Step 5: Run the container

```bash
docker run -p 3000:3000 myapp
```

`-p 3000:3000` = connect port 3000 on my laptop to port 3000 inside the container. Open `http://localhost:3000`.

Step 6: Check and debug

```bash
docker ps                # is it running?
docker logs <container> # what is it printing? any errors?
```

Step 7: Stop and clean up

```bash
docker stop <container>
docker rm <container>
```

Step 8 (optional): Push to Docker Hub so others (or a server) can pull it

```bash
docker login
docker tag myapp yourusername/myapp:1.0
docker push yourusername/myapp:1.0
```

---

4. Docker boilerplate (copy and edit)

4.1 Node.js / Express (MERN backend)

```dockerfile
```

```
# Start from a small image that already has Node installed
FROM node:20-alpine

# All next commands run inside /app in the container
WORKDIR /app

# Copy ONLY dependency files first (so Docker can cache npm install)
COPY package*.json ./

# Install dependencies (happens at BUILD time)
RUN npm ci

# Now copy the rest of the code
COPY . .

# Tell people which port the app uses (documentation only)
EXPOSE 3000

# Command that runs when the container STARTS
CMD ["node", "index.js"]
```

4.2 Python (Flask / FastAPI / scripts)

dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["python", "app.py"]
```

4.3 React frontend (multi-stage, final image is tiny)

dockerfile

```
# Stage 1: build the React app
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: serve the built files with nginx (no Node needed in final image)
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Note: Vite builds to `dist`, Create React App builds to `build`. Change the path if needed.

4.4 `.dockerignore`

```
node_modules
.git
.env
*.log
dist
build
__pycache__
.DS_Store
```

4.5 `docker-compose.yml` (MERN: frontend + backend + MongoDB)

yaml

```yaml
services:
  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      - MONGO_URL=mongodb://db:27017/mydb   # "db" = service name below
    depends_on:
      - db

  db:
    image: mongo:7
    volumes:
      - dbdata:/data/db      # keeps data after container stops

volumes:
  dbdata:
```

Containers in the same compose file talk to each other **by service name** (`db`, `backend`), not `localhost`.

bash

```bash
docker compose up -d --build   # build and start everything in background
docker compose logs -f         # watch logs live
docker compose down            # stop and remove (data in volume stays)
```

4.6 Good habits

- Copy dependency files first, then code (faster rebuilds thanks to caching)
- Never put passwords or API keys in the Dockerfile. Use env variables or secrets.
- Use small base images (`alpine`, `slim`)
- Pin versions (`node:20-alpine`, not `node:latest`)
- One container = one job

---

5. Useful Docker commands

Images

```bash
docker images                    # list images on my machine
docker build -t name:tag .       # build image
docker pull nginx                # download image
docker rmi <image>               # delete image
docker image prune               # delete unused (dangling) images
docker history <image>           # see layers and sizes
```

Containers: run

```bash
docker run <image>                          # run once
docker run -d <image>                       # -d = background
docker run -p 8080:80 <image>               # port mapping HOST:CONTAINER
docker run --name mycont <image>            # give it a name
docker run --rm <image>                     # auto-delete when it stops
docker run -it ubuntu bash                  # interactive terminal inside
docker run -e KEY=value <image>             # pass an env variable
docker run -v mydata:/data <image>          # attach a volume
docker run -v $(pwd):/app <image>           # mount current folder (live code edits
```

Containers: manage

```bash
docker ps                 # running containers
docker ps -a              # all containers, including stopped
docker stop <name>        # stop nicely
docker start <name>       # start a stopped one
docker restart <name>
docker rm <name>          # delete container
docker rm -f <name>       # force delete (even if running)
```

Debug (most used when something breaks)

```bash
```

```
docker logs <name>              # see output
docker logs -f <name>           # follow live
docker exec -it <name> sh       # open a shell INSIDE a running container
docker inspect <name>           # full details (IP, ports, env, mounts)
docker stats                    # live CPU and RAM usage
docker cp <name>:/app/file.txt .   # copy a file out of a container
```

Volumes and networks

bash

```
docker volume ls
docker volume rm <volume>
docker network ls
```

Clean up (when disk is full)

bash

```
docker system df                # how much space Docker uses
docker system prune             # remove stopped containers, unused networks, dangl
docker system prune -a          # also remove ALL unused images (careful!)
```

Compose

bash

```
docker compose up               # start all services
docker compose up -d --build    # rebuild and start in background
docker compose down             # stop and remove
docker compose down -v          # also delete volumes (data is gone!)
docker compose ps               # status of services
docker compose logs -f <svc>    # logs of one service
docker compose exec <svc> sh    # shell inside a service
```

---

6. CI/CD: what it is

CI = Continuous Integration Every time I push code, a robot automatically **builds and tests** it. Like a teacher who checks your homework the second you submit it.

CD = Continuous Delivery / Deployment If tests pass, the robot automatically **ships** the app (to a server, or Docker registry).

- *Delivery*: ready to ship, a human presses the button.
- *Deployment*: ships automatically, no human.

The pipeline (a conveyor belt)

```
push code → build → test → build Docker image → push image → deploy
```

If any stage fails, the belt stops and I get a red ❌.

GitHub Actions words

Word	Easy meaning
Workflow	One automation file (YAML) in `.github/workflows/`
Trigger (`on`)	What starts it: push, pull request, schedule
Job	A group of steps on one machine
Step	One task: run a command or use an action
Runner	The machine that runs the job (`ubuntu-latest`)
Action	Ready-made step by others (`actions/checkout`)
Secrets	Safe storage for passwords/tokens

---

7. When to use / when NOT to use CI/CD

✅ Use CI/CD when

- You work in a team and many people push code
- You have tests and want them to run automatically
- You deploy often and want it automatic and repeatable
- You want a green tick on your GitHub repo (looks professional on resume too)
- You build Docker images regularly

❌ Don't bother when

- It's a quick throwaway experiment
- Solo tiny project with no tests and no deployment
- You have no tests yet. A pipeline with nothing to check is pointless, so write tests first.
- You haven't learned Git/GitHub basics yet

Rule of thumb: if you find yourself repeating "run tests, build, upload, deploy" by hand, automate it.

---

8. CI/CD: steps to set up (GitHub Actions)

Step 1: Put your project on GitHub.

Step 2: Make the folder `.github/workflows/` in the repo root. (Exact name, with the dot.)

Step 3: Create a file inside, like `ci.yml`. Write the workflow (boilerplate below).

Step 4: Add secrets if needed (Docker Hub login, server keys): Repo → Settings → Secrets and variables → Actions → New repository secret.

Step 5: Commit and push.

```bash
git add .github/workflows/ci.yml
git commit -m "add CI pipeline"
git push
```

Step 6: Open the **Actions** tab on GitHub. Watch it run. 🟡 running, ✅ passed, ❌ failed.

Step 7: If it fails, click the run → click the red step → read the log. The error is almost always in the last red step.

Step 8 (bonus): Add a badge to README so everyone sees the status:

```markdown
![CI](https://github.com/USERNAME/REPO/actions/workflows/ci.yml/badge.svg)
```

---

9. CI/CD boilerplate (copy and edit)

9.1 CI for Node.js (build + test)

`.github/workflows/ci.yml`

```yaml
```

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # download my code onto the runner

      - uses: actions/setup-node@v4        # install Node
        with:
          node-version: 20
          cache: npm                       # cache node_modules for speed

      - run: npm ci                        # clean install
      - run: npm run build --if-present    # build (skips if no build script)
      - run: npm test                      # run tests; failure = red ❌
```

9.2 CI for Python

yaml

```yaml
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest
```

9.3 Build and push Docker image (CD part)

`.github/workflows/docker.yml`

yaml

```yaml
name: Docker

on:
  push:
    branches: [main]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: yourusername/myapp:latest
```

Make the Docker Hub token at hub.docker.com → Account Settings → Security → New Access Token. Save it as the secret `DOCKERHUB_TOKEN`.

9.4 Test first, then build image (jobs in order)

yaml

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm test

  docker:
    needs: test                  # only runs if "test" passed
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: yourusername/myapp:latest
```

---

10. Common errors and fixes

Problem	Likely reason	Fix
App runs but can't open in browser	Wrong port mapping	Check `-p HOST:CONTAINER`, and the app must listen on `0.0.0.0`, not `localhost`
`port is already allocated`	Something else uses that port	Stop it, or use a different host port (`-p 3001:3000`)
Backend can't reach database in compose	Using `localhost`	Use the service name (`db`) instead
Data disappeared	No volume	Add a volume
`COPY failed: file not found`	File is outside build context or ignored	Check path and `.dockerignore`
Image is huge	Big base image, copied `node_modules`	Use `alpine` / `slim`, add `.dockerignore`, use multi-stage
Changes not showing	Using old image	Rebuild: `docker build` or `docker compose up --build`
`permission denied` (Linux)	User not in docker group	`sudo usermod -aG docker $USER`, then log out and in
Pipeline not starting	Wrong folder/file name or YAML indentation	Must be `.github/workflows/*.yml`, use spaces not tabs
`Error: Process completed with exit code 1`	A command failed	Open the red step, read the log above that line
Login to Docker Hub fails in CI	Secret name typo or wrong token	Recheck secret names exactly
Disk full	Old images/containers piling up	`docker system prune`

11. My project notes

Fill this in so future me remembers.

- **Project name:**
- **What I put in Docker (frontend / backend / DB):**
- **Which commands I used the most:**
- **What broke and how I fixed it:**
- **Did we have CI/CD? What did the pipeline do:**

---

12. To learn next

- [ ] Multi-stage builds in more depth
- [ ] Docker networks
- [ ] Docker secrets and env files
- [ ] Deploy to a server / cloud (AWS, Render, etc.)
- [ ] Caching in GitHub Actions
- [ ] Dev / staging / production deploys
- [ ] Docker with hardware / robotics (ROS 2 containers)